

	0x0_	0x1_	0x2_	0x3_	0x4_	0x5_	0x6_	0x7_	0x8_	0x9_	0xA_	0xB_	0xC_	0xD_	0xE_	0xF_
0x_0	NOP	MACX,0	MACX,16	RREG,0	RREG,8	FSEL,D	CSF,RD	CAF,RD	WREG,UI	AND	PASS	FORK,RF	ROT,RF	HOP,RF	GPNT	
0x_1	LDB	MACX,1	MACX,17	WREG,0	WREG,8	FSEL,F	CSF,RR	CAF,RR	WREG,B1	OR	NOT	FORK,RR	ROT,RR	HOP,RR	GREL	
0x_2	WRMAC	MACX,2	MACX,18	RREG,1	RREG,9	FSEL,Z	CSF,RL	CAF,RL	WREG,B2	XOR	INC	FORK,RL	ROT,RL	HOP,RL	ACT	
0x_3	CLRL	MACX,3	MACX,19	WREG,1	WREG,9	FSEL,N	CSF,LB	CAF,LB	WREG,AC	IMP	DEC	FORK,RB	ROT,RB	HOP,RB	SCAN	
0x_4	MACX,L	MACX,4	MACX,20	RREG,2	RREG,A	FSEL,V	CSF,JMP	CAF,JMP	RREG,AC	NAND	NEG	FORK,AF	ROT,AF	HOP,AF	MINE	
0x_5	BKPT	MACX,5	MACX,21	WREG,2	WREG,A	FSEL,C	CSF,JSR	CAF,JSR	RREG,U0	NOR	WSW	FORK,AR	ROT,AR	HOP,AR		
0x_6		MACX,6	MACX,22	RREG,3	PVREG	FSEL,M	CSF,RET	CAF,RET	RREG,B0	XNOR	BRR	FORK,AL	ROT,AL	HOP,AL		
0x_7	END	MACX,7	MACX,23	WREG,3	GPREG	FSEL,W	CSF,END	CAF,END	CLRA	INH	BRL	FORK,AB	ROT,AB	HOP,AB		
0x_8	RREG,Z	MACX,8	MACX,24	RREG,4	CLRF	FSEL,0	CST,RD	CAT,RD	LUON	BSET	ROR	FORK,MA	REDO	MREAD		
0x_9	WREG,DP	MACX,9	MACX,25	WREG,4	SETF	FSEL,K	CST,RR	CAT,RR	LBON	BCLR	ROL	TERM	LOOP	MWRITE		
0x_A	RREG,LL	MACX,10	MACX,26	RREG,5	CLRC	FSEL,X	CST,RL	CAT,RL	LOFF	BINV	RRX	DUPE	LRET	WEMEM		
0x_B	WREG,LL	MACX,11	MACX,27	WREG,5	SETC	FSEL,R	CST,LB	CAT,LB	ACOFF	BTST	RLX	TRMD	LBRK	WPMEM		
0x_C	RREG,UL	MACX,12	MACX,28	RREG,6	CLRV	FSEL,A	CST,JMP	CAT,JMP	ACON	ADD	LSR	RREG,AP	JMP	RSTMA		
0x_D	WREG,UL	MACX,13	MACX,29	WREG,6	SETV	FSEL,B	CST,JSR	CAT,JSR		CMP	LSL	RREG,EP	JSR	OLMA		
0x_E		MACX,14	MACX,30	RREG,7	FDSET	FSEL,P	CST,RET	CAT,RET		SUB	ASR		RET	IJMP		
0x_F		MACX,15	MACX,31	WREG,7	FDCLR	FSEL,T	CST,END	CAT,END		SUB,R			RREG,PC			

0001

0010

0011

0100

0101

0110

0111

1xxx

7654 3210	Instruction Bit Decoding
000- ----	Instruction proc 1
0010 ----	Instruction proc 2
0011 ----	Context switcher
0100 ----	Program counter
0101 ----	ALU opcodes 1
011- ----	ALU opcodes 2
1000 ----	Memory/addr. opcodes
1001 ----	Register opcodes 1
1010 0---	Register opcodes 2
1010 1---	Flag opcodes 1
1011 ----	Flag opcodes 2
110- ----	Conditional opcodes
1110 ----	Game opcodes
1111 ----	Unimplemented

Quick Cond. Opcodes	
0x0	REDO
0x1	ROT,RR
0x2	ROT,RL
0x3	LBRK
0x4	JMP
0x5	JSR
0x6	RET
0x7	END

Old Module Network Addresses	
0000	= Instruction Processor
0001	= Context Switcher
0010	= Program Counter Control
0011	= ALU Subsystem
0100	= Memory Bank
0101	= Register Bank
0110	= Conditional & Flag Subsystem
0111	= Game Feature Subsystem
1000	=
1001	=
1010	=
1011	=
1100	=
1101	=
1110	=
1111	=

Module Network Addresses	
0001	= Instruction Processor
0010	= Register Bank
0011	= Conditional & Flag Subsystem
0100	= ALU Subsystem
0101	= Context Switcher
0110	= Program Counter Control & Memory
0111	= Game Feature Subsystem
1000	=
1001	=
1010	=
1011	=
1100	=
1101	=
1110	=
1111	=

ALU configuration (Use RREG,AC & WREG,AC to read and update)
Default configuration is all zero values.
Byte 0: Basic ALU and auto-output configuration
b0-1: ALU configuration update method:
00=write: write 30-bit input value to ALU config register
01=OR: for each input bit set, set corresponding config bit
10=INH: for each input bit set, clear corresponding config bit
11=XOR: for each input bit set, toggle corresponding config bit
b2: ALU_U width:0=32-bit,1=8-bit
b3: ALU_B width:0=32-bit,1=8-bit
b4: ALU_U registers 0=persistent/1=temporary
b5: ALU_B registers 0=persistent/1=temporary
b6: Disable ALU_U0 auto-output T/F
b7: Disable ALU_B0 auto-output T/F
Byte 1: Flag configuration
b8-9: C/V flag source: 00=auto,01=ALU_U0,10=ALU_B0,11=ignore
b10-11: PNZ flag source: 00=auto,01=ALU_U0,10=ALU_B0,11=ignore
b12-13: ALU_U auto flag select:00=Z,01=C,10=none,11=V
b14-15: ALU_B auto flag select:00=Z,01=C,10=none,11=V
Byte 2: ALU input/crossover configuration
b16-17: ALU_UI Auto-in: 00=none,01=ALU_U0,10=ALU_B0,11=Active Out
b18-19: ALU_B1 Auto-in: 00=none,01=ALU_U0,10=ALU_B0,11=Active Out
b20-21: ALU_B2 Auto-in: 00=none,01=ALU_U0,10=ALU_B0,11=Active Out
b22-23: ALU_U1 auto-CMD* when binary ALU opcode is executed.
00=off, 01=auto-INC, 10=auto-DEC, 11= <i>unimplemented</i> .
Byte 3: Unused
b24-31: Unused
* auto-CMD hijacks the use of reported C or V flag states for INC/DEC overflow/underflow conditions, but internally, the binary ALU still maintains its own carry and overflow flag bits for its operations. It just will not report C or V flag states while auto-CMD is active.

TODO/WIP: None.

Read/Write	Register List	Read-Only
Register 0 – General Purpose 0		Register Z – Static 0x00000000
Register 1 – General Purpose 1		
Register 2 – General Purpose 2	Register XY – Memory XY Rel. Address	
Register 3 – General Purpose 3		
Register 4 – General Purpose 4	Register AP – Active/Dupe Process Counts	
Register 5 – General Purpose 5	Register PC – Program Counter	
Register 6 – General Purpose 6	Register EP – Extended Process Info.	
Register 7 – General Purpose 7		
Register 8 – General Purpose 8	Register UO – ALU_UO Output Register	
Register 9 – General Purpose 9	Register AO – ALU_BO Output Register	
Register A – Group Register A		
	Write-Only	
Register AC – ALU Config Register		Register DP – Display Register
Register MD – Memory Data Access	Register UI – ALU_UI Input Register	
	Register B1 – ALU_B1 Input Register	
Register LL – Lower LOAD Register	Register B2 – ALU_B2 Input Register	
Register UL – Upper LOAD Register		

Process Control Block Memory (Per-Process)
Default 32-bits each unless indicated
0001 – Lower LOAD Register
0001 – Upper LOAD Register
0001 – Macro Addr (5b + 3b dummy)
0001 – Macro 0 (8B)
0001 – Macro 1 (8B)
0001 – Macro 2 (8B)
0001 – Macro 3 (8B)
0001 – Display Register
0001 – Temp. Register 1
0010 – General Purpose 0
0010 – General Purpose 1
0010 – General Purpose 2
0010 – General Purpose 3
0010 – General Purpose 4
0010 – General Purpose 5
0010 – General Purpose 6
0010 – General Purpose 7
0010 – General Purpose 8
0010 – General Purpose 9
0010 – Temp. Register 2
0010 – Pvt./Grp. Config Bitmap (2B)
0011 – Flag Storage + Flag def. (2B)
0011 – Flag Select S + A (1B)
0011 – Temp. Register 3
0100 – ALU_UI Input Register
0100 – ALU_UO Output Register
0100 – ALU_B1 Input Register
0100 – ALU_B2 Input Register
0100 – ALU_BO Output Register
0100 – ALU Config Register
0100 – Temp. Register 4
0101 – Proc. Status & Dupe Count
0110 – Program Counter (12B total)
0110 – JSR Return Address
0110 – Loop Return Address
0110 – Memory Data Access
0110 – Memory XY Rel. Address
0110 – Temp. Register 6
0111 – Game Resource Tracking
1111 – Dummy (1B)

Module Network Addresses
0001 = Instruction Processor
0010 = Register Bank
0011 = Conditional & Flag Subsystem
0100 = ALU Subsystem
0101 = Context Switcher
0110 = Program Counter Control & Memory
0111 = Game Feature Subsystem
1000 =
1001 =
1010 =
1011 =
1100 =
1101 =
1110 =
1111 =

	Bits 18-31	Bits 16-17	Bits 8-15	Bits 0-7
Register PC Program Counter	Zeroes	PC Direction 0-3	Program Counter Y Address 0-255	Program Counter X Address 0-255

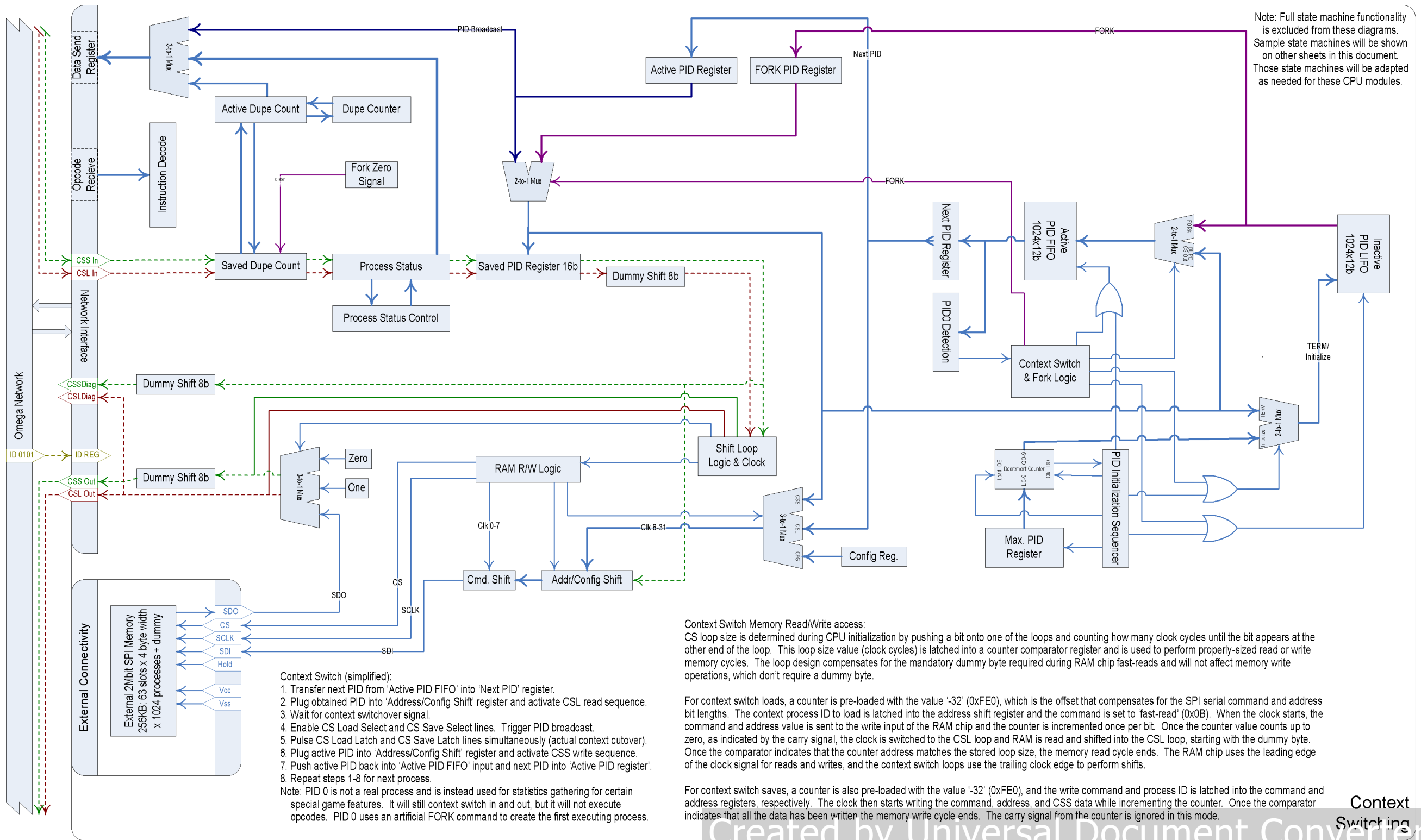
	Bits 26-31	Bits 16-25	Bits 10-15	Bits 0-9
Register 0x1D Active Process Count	Zeroes	Maximum UID Number 1-1023	Zeroes	User Process Count 1-1023

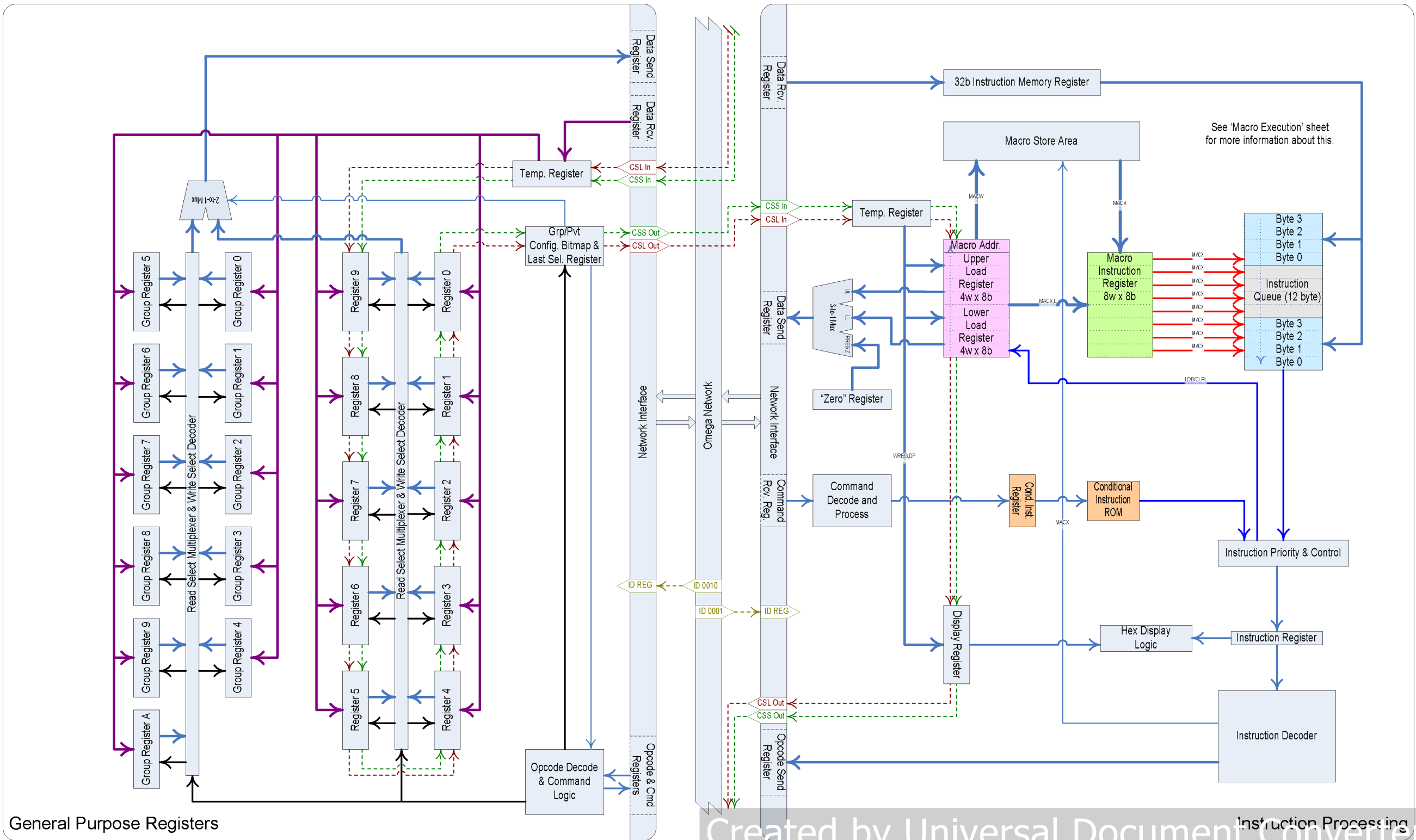
	Bits 24-31	Bits 8-23	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
PCB Extended Process Info	Process duplicate Count 0-255 (primary)	Empty								
		unimplemented								
		unimplemented								
		unimplemented								
		unimplemented								
		0 = off, 1 = Process has underlay addressing enabled.								
		0 = off, 1 = Process is an underlay process.								
		0 = off, 1 = Process is currently strong-faulted.								
		0 = off, 1 = Process is terminated.								

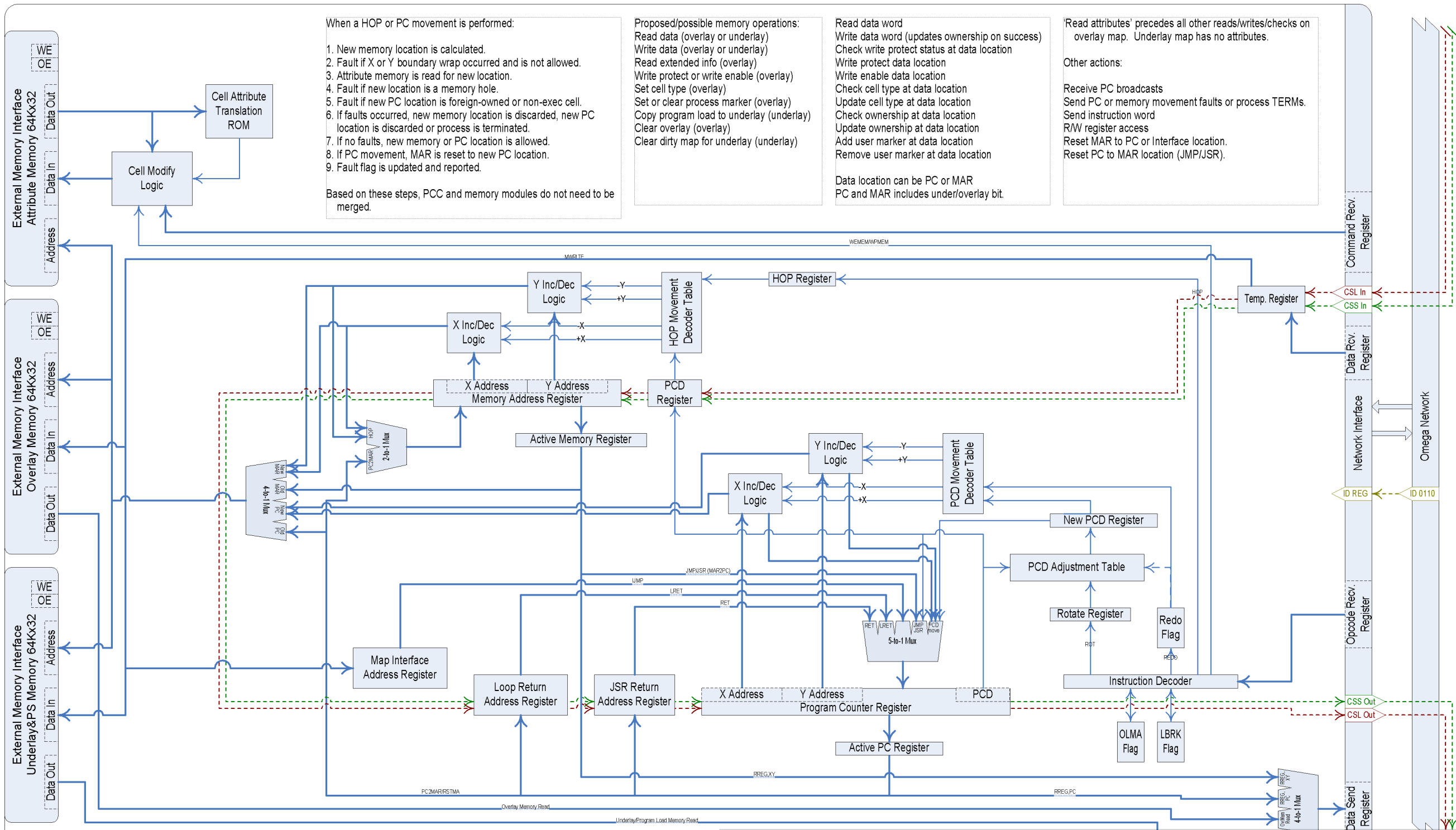
	Bits 16-31	Bits 8-15	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
Register Aux. Memory Data Register (no longer visible)	Occupation Markers 0-15	Zeroes			00 = Unowned Cells	Special Block T/F	TERM on PC Contact T/F		MAR contact: 00=Normal 01=F.Fault 10=S.Fault 11=Barrier	
					01 = write enable 10 = interface 11 = write protect					Cell Owner 0-15

b0-b3 = character
b4 = U0 highlight color or background
b5 = U1 highlight color or background
b6 = U2 highlight color or background
b7 = U3 highlight color or background
3b char color:
000 background (black or user)
001 foreground (gray)
010 random color 1
011 random color 2
100 U0 selective color (red)
101 U1 selective color (green)
110 U2 selective color (blue)
111 U3 selective color (yellow)

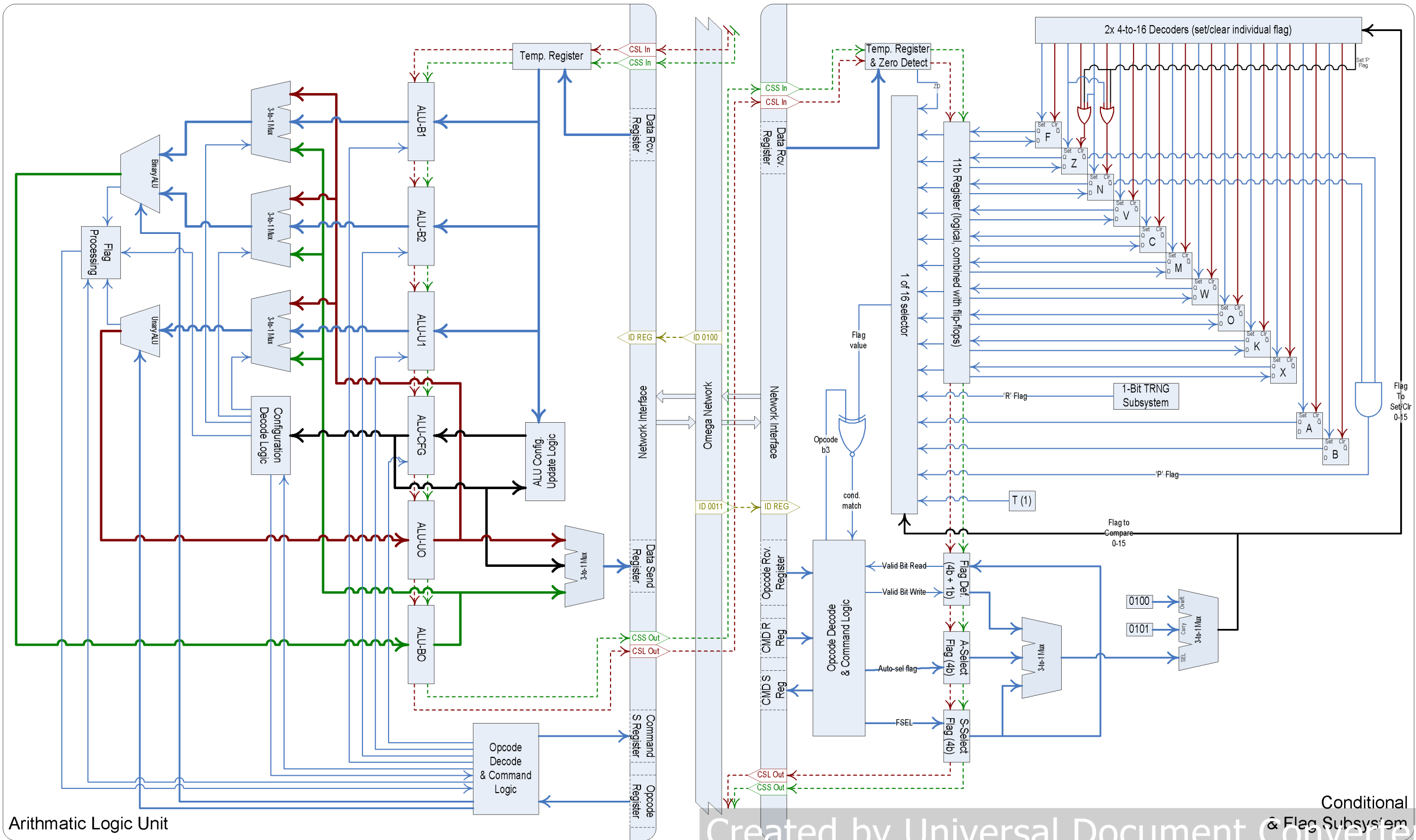
0000 = Empty Cell	0100 = Mined
0001 = Glitch Cell	0101 = Trapped
0010 = Corrupt Cell	0110 = Corrupt Cell
0011 = Memory Hole	0111 = N/A







Program Counter Control & Memory



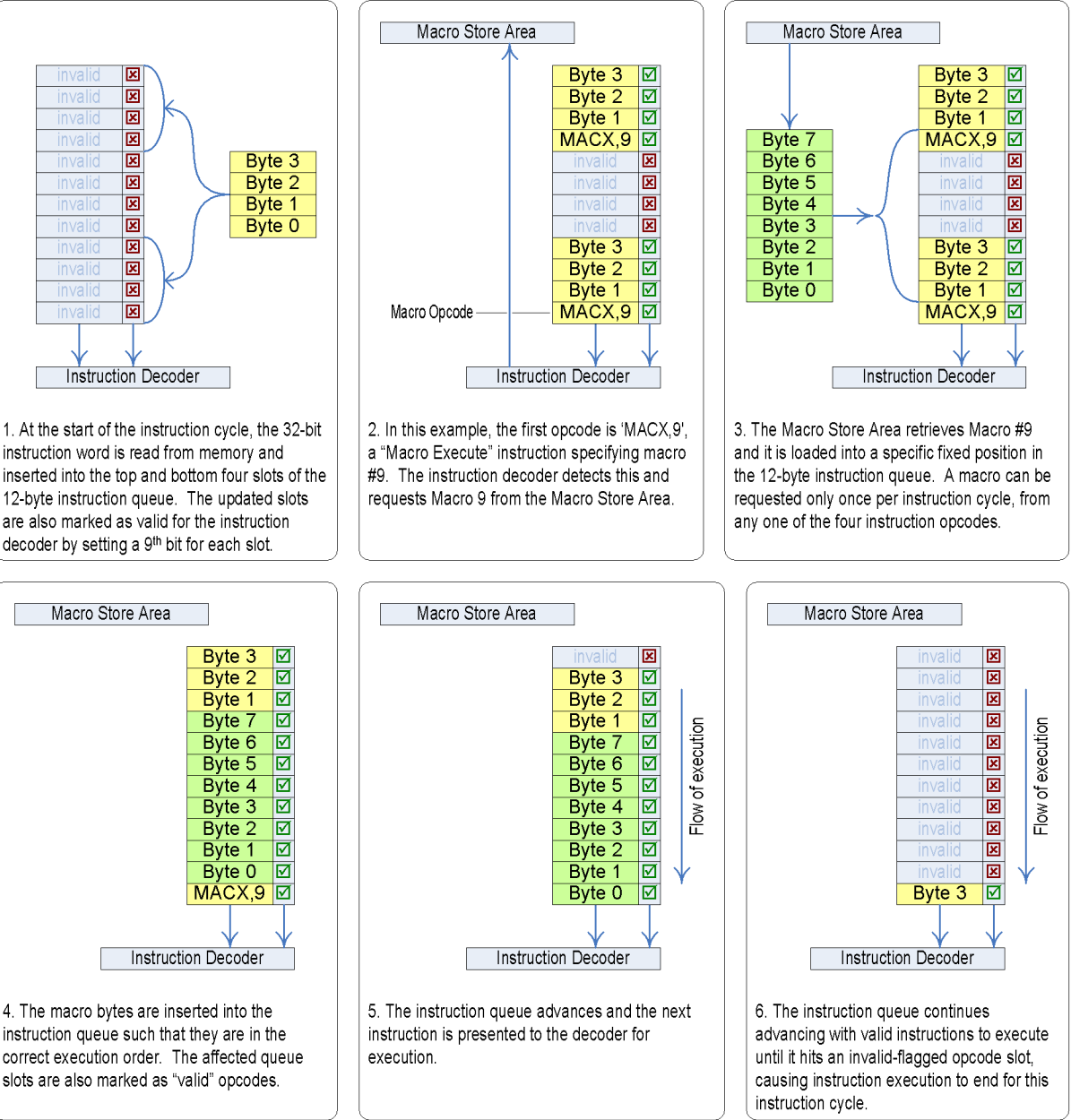
Macro Execution Example

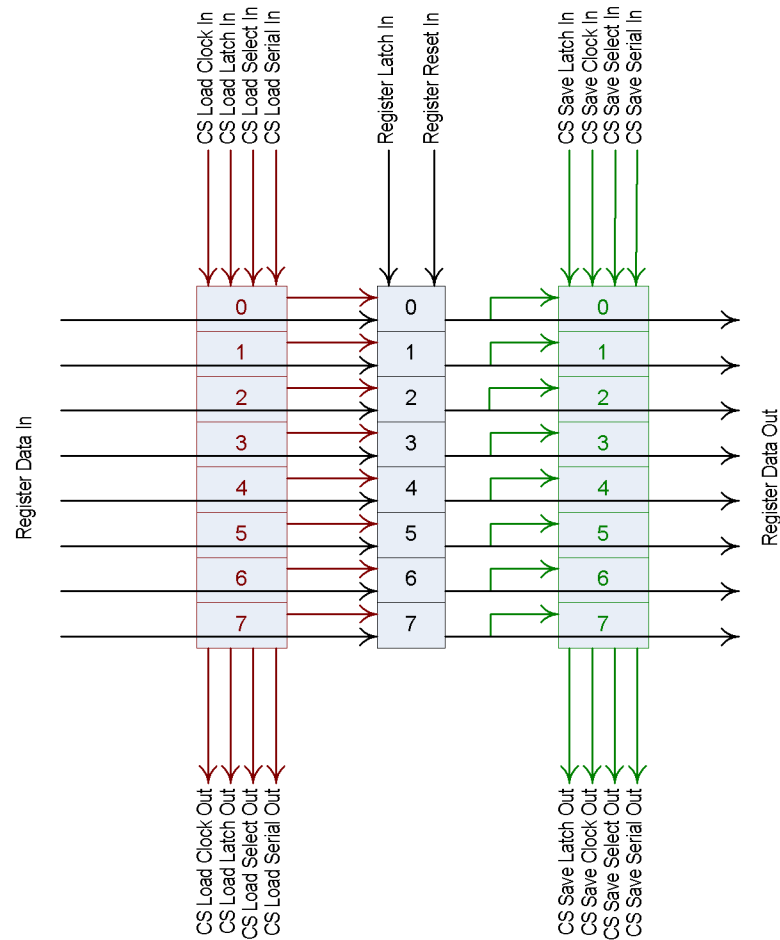
32-bit instruction word:

Byte 3	Byte 2	Byte 1	Byte 0
--------	--------	--------	--------

64-bit macro:

Byte 7	Byte 6	Byte 5	Byte 4	Byte 3	Byte 2	Byte 1	Byte 0
--------	--------	--------	--------	--------	--------	--------	--------





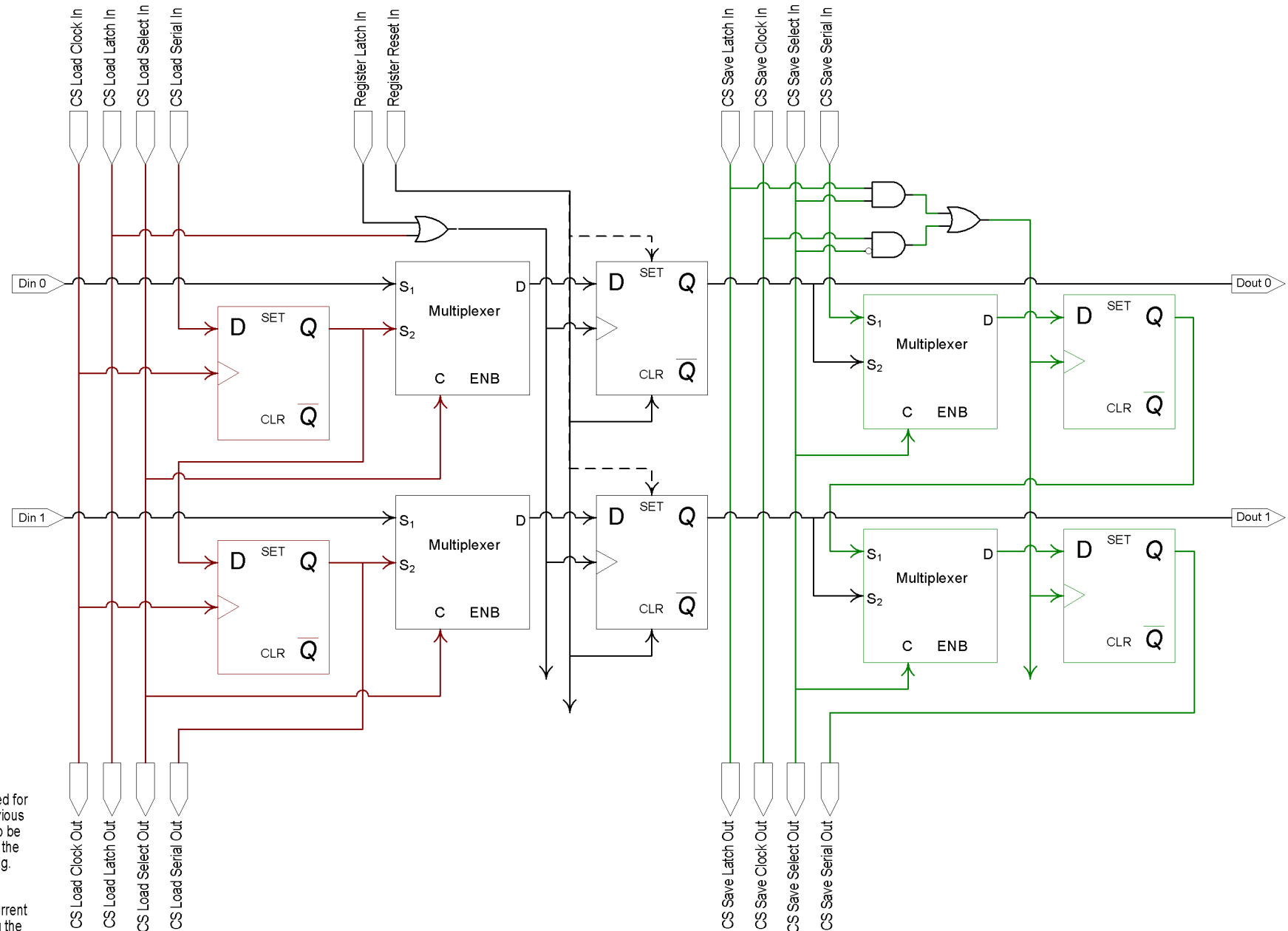
This register concept is used for "saveable" CPU registers that need to be stored and loaded during the context switching of processes.

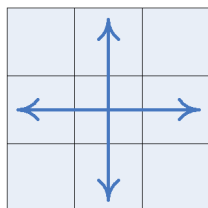
The register consists of a regular parallel I/O register attached to two serial shift registers – the first one (in red) used for readying the next process context to be loaded in, and the second one (in green) to be used for saving out the previous process context. The dual latch lines connected to the parallel register direct multiplexers to select which input is to be latched, and the CS latch lines carry out the actual latching of data into the register. The CS clock lines are pulsed the exact number of data shifts needed to transfer the serial data and only operate when actual data shifting is occurring. The speed of these clock pulses is based on the timing requirements of the process control block (PCB) memory.

For this register, a context switch is performed by bringing the CS Load/Save Select lines high, then latching the current parallel register into the "save" context switch (green) using the CS_Save_Latch signal and simultaneously latching the new processor data from the "load" context switch (red) into the parallel register using the CS_Load_Latch signal. While the new process is active, the context data from the now inactive process can then be shifted out and stored at the same time as the next process context is read and shifted into the "load" context switch. Part of this context switch functionality is also used for duplicating the register image of processes being forked.

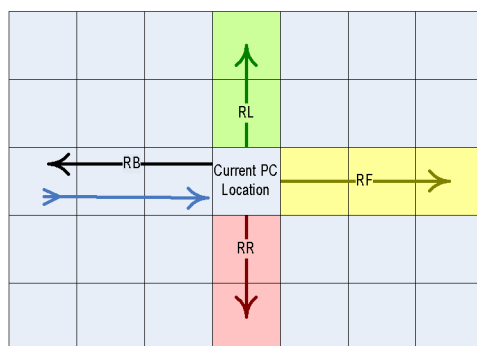
This approach minimizes the idle time of the CPU waiting for the context switch to complete.

Shown above: 8-bit simplified block diagram on the left, and a 2-bit "slice" of a standard logic diagram on the right.



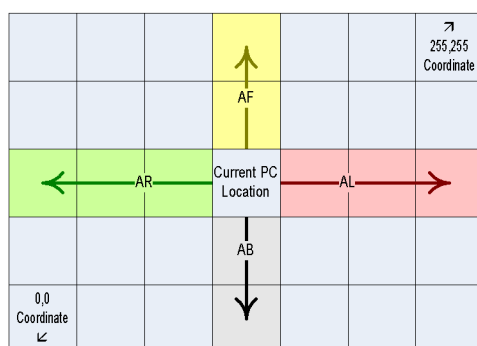


Execution Directions



Relative Directions

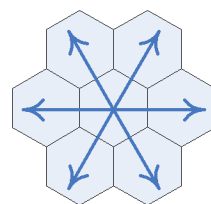
Blue Arrow: Current execution direction
Yellow Arrow: Relative Forward (RF)
Green Arrow: Relative Left (RL)
Red Arrow: Relative Right (RR)
Black Arrow: Relative Back (RB)



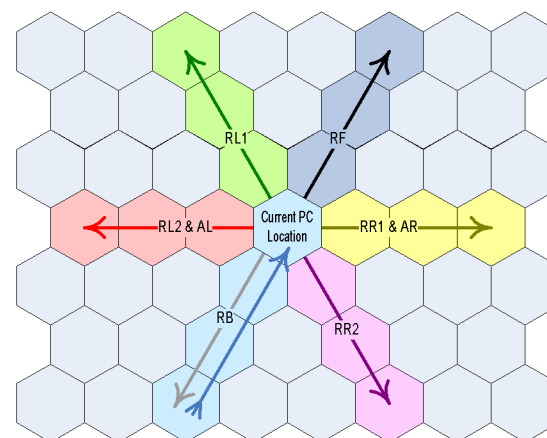
Absolute Directions

Yellow Arrow: Absolute Forward (AF)
Black Arrow: Absolute Back (AB)
Green Arrow: Absolute Right (AR)
Red Arrow: Absolute Left (AL)

Square Grid

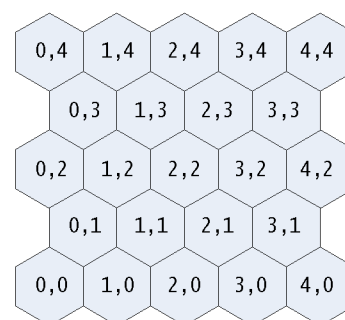


Execution Directions

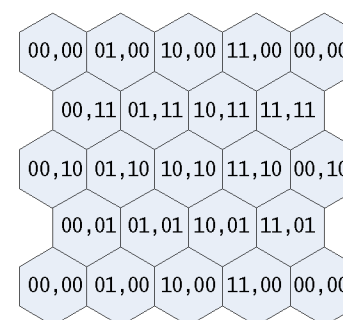


Relative and Absolute Directions

Blue Arrow: Current execution direction
Black Arrow: Relative Forward (RF)
Gray Arrow: Relative Backward (RB)
Yellow Arrow: Relative Right 1 (RR1) and Absolute Right (AR)
Purple Arrow: Relative Right 2 (RR2)
Green Arrow: Relative Left 1 (RL1)
Red Arrow: Relative Left 2 (RL2) and Absolute Left (AL)

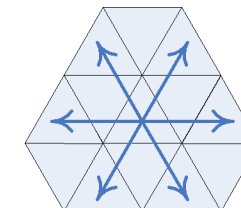


Coordinate System

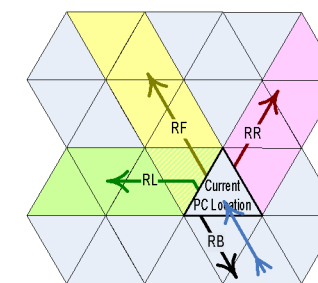


Coordinate System (Binary)

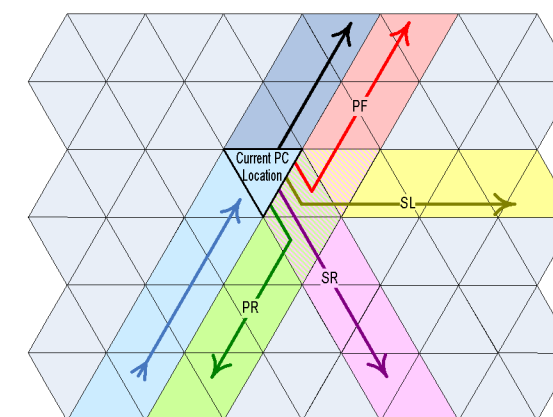
Hexagonal Grid



Execution Directions

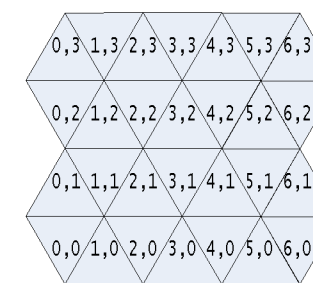


Relative Directions

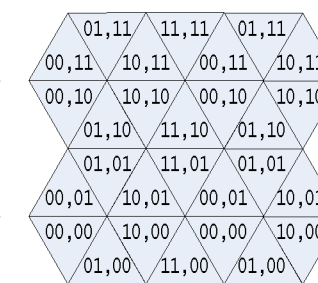


Parallel & Side Directions

Blue Arrow: Current execution direction
Gold Arrow: Side Left (SL)
Purple Arrow: Side Right (SR)
Red Arrow: Parallel Forward (PF)
Green Arrow: Parallel Reverse (PR)
Black Arrow: No direction change

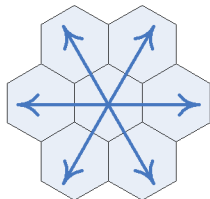


Coordinate System

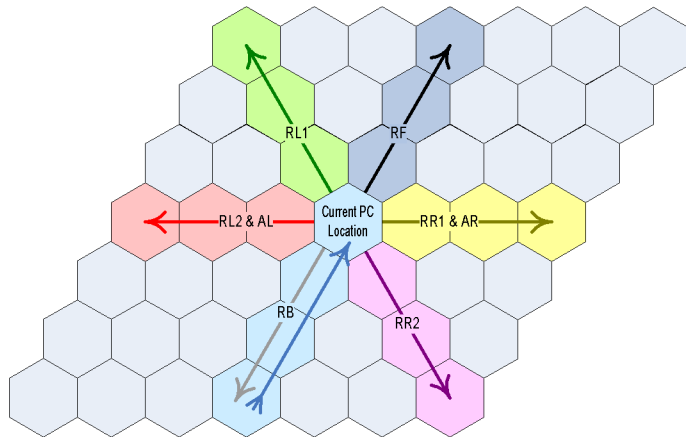


Coordinate System (Binary)

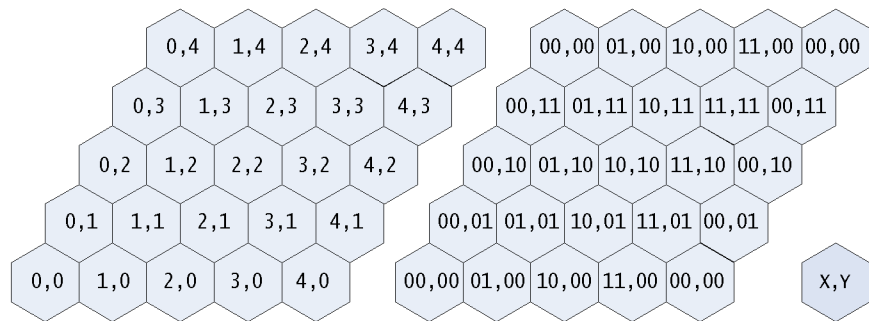
Triangle Grid



Execution Directions



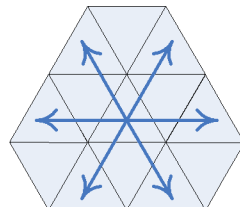
Relative and Absolute Directions
Blue Arrow: Current execution direction
Black Arrow: Relative Forward (RF)
Gray Arrow: Relative Backward (RB)
Yellow Arrow: Relative Right 1 (RR1) and Absolute Right (AR)
Purple Arrow: Relative Right 2 (RR2)
Green Arrow: Relative Left 1 (RL1)
Red Arrow: Relative Left 2 (RL2) and Absolute Left (AL)



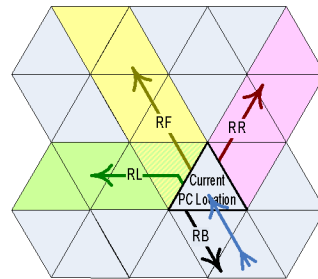
Coordinate System

Coordinate System (Binary 2LSB)

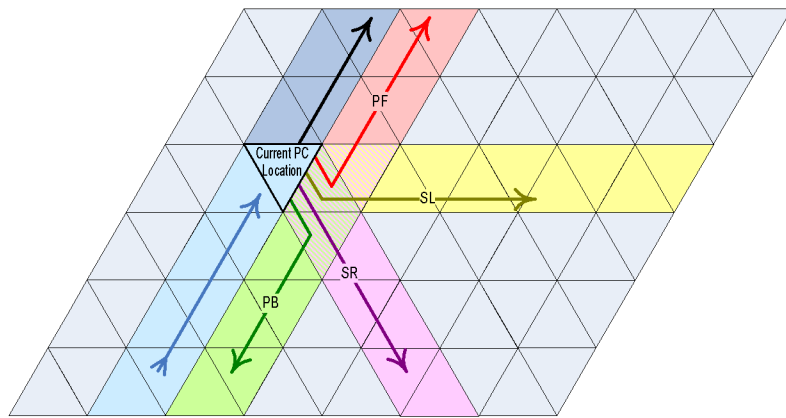
Hexagonal Grid (Alternate)



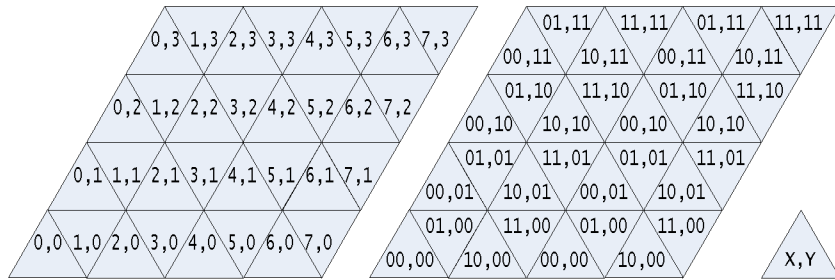
Execution Directions



Relative Directions



Parallel & Side Directions
Blue Arrow: Current execution direction
Gold Arrow: Side Left (SL)
Purple Arrow: Side Right (SR)
Red Arrow: Parallel Forward (PF)
Green Arrow: Parallel Back (PB)
Black Arrow: No direction change



Coordinate System

Coordinate System (Binary 2LSB)

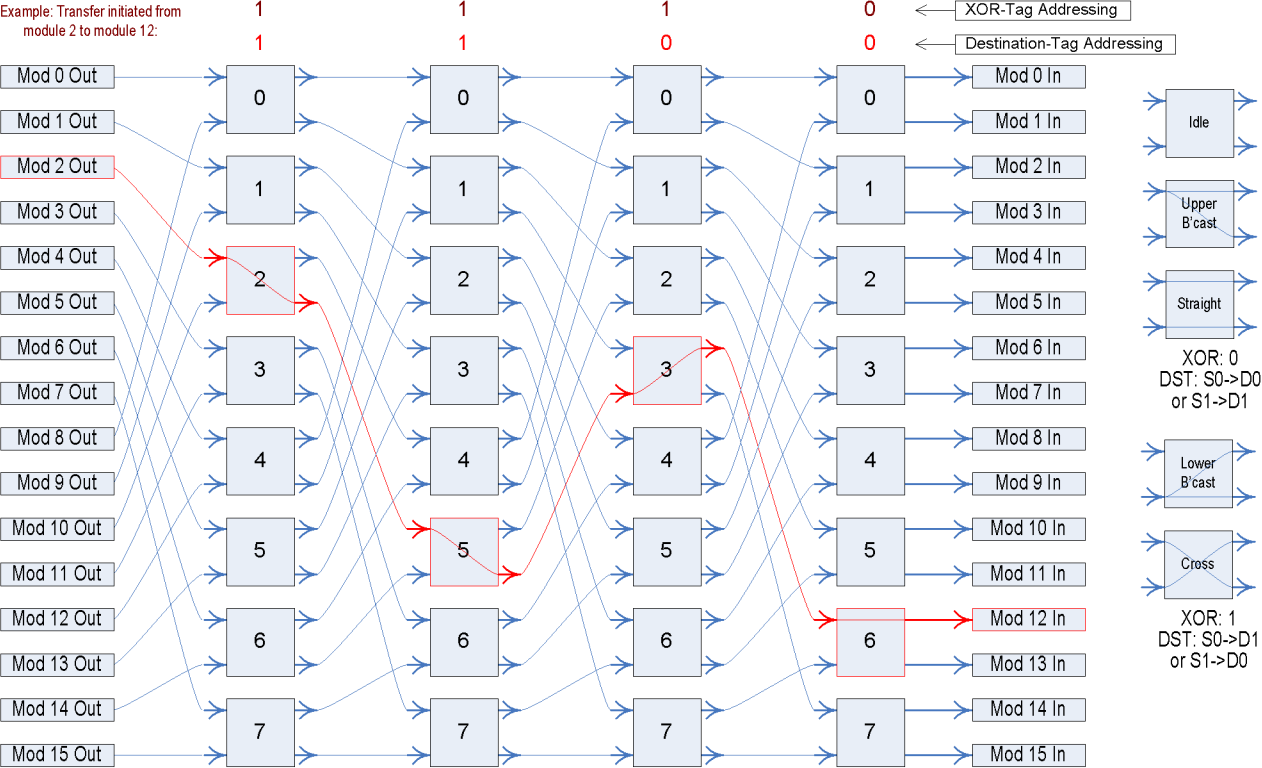
Triangle Grid (Alternate)

Hexagonal Grid	0xB_	0xC_	0xD_
0x_0	FORK, RF	ROT, RF	HOP, RF
0x_1	FORK, RR	ROT, RR	HOP, RR
0x_2	FORK, RR2	ROT, RR2	HOP, RR2
0x_3	FORK, RL	ROT, RL	HOP, RL
0x_4	FORK, RL2	ROT, RL2	HOP, RL2
0x_5	FORK, RB	ROT, RB	HOP, RB
0x_6	FORK, AR	ROT, AR	HOP, AR
0x_7	FORK, AL	ROT, AL	HOP, AL

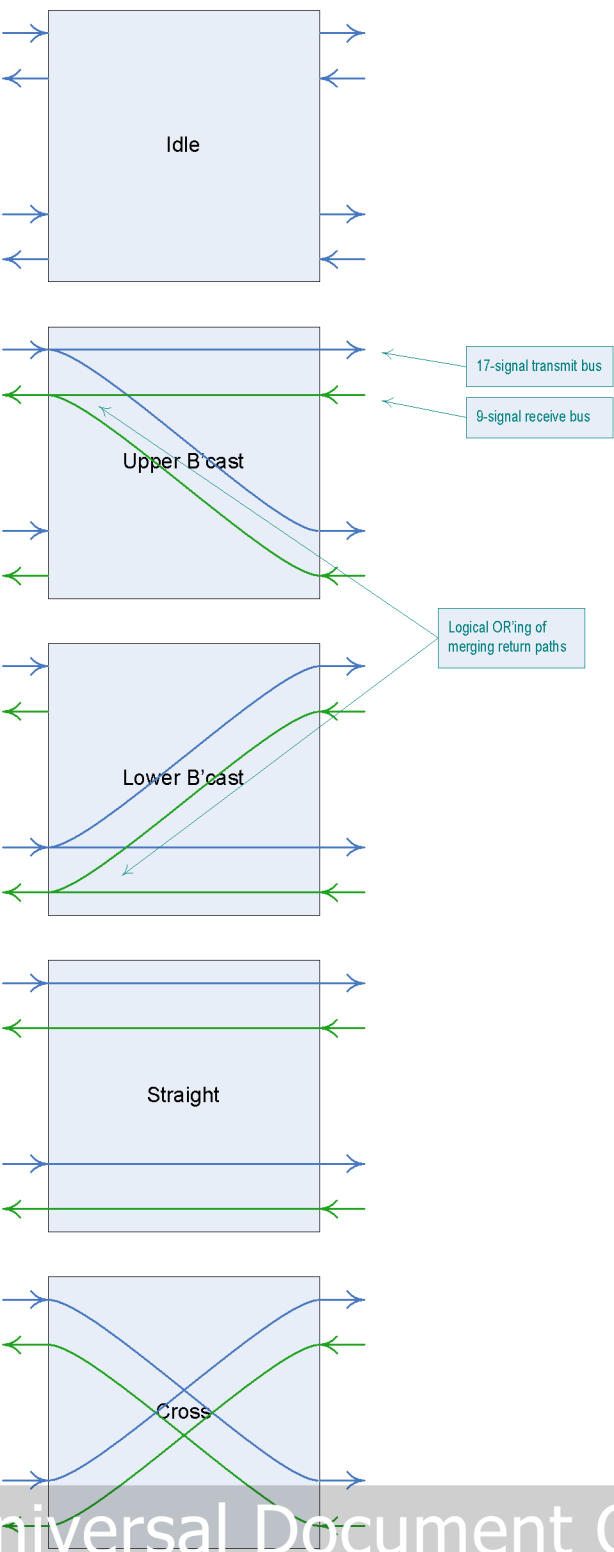
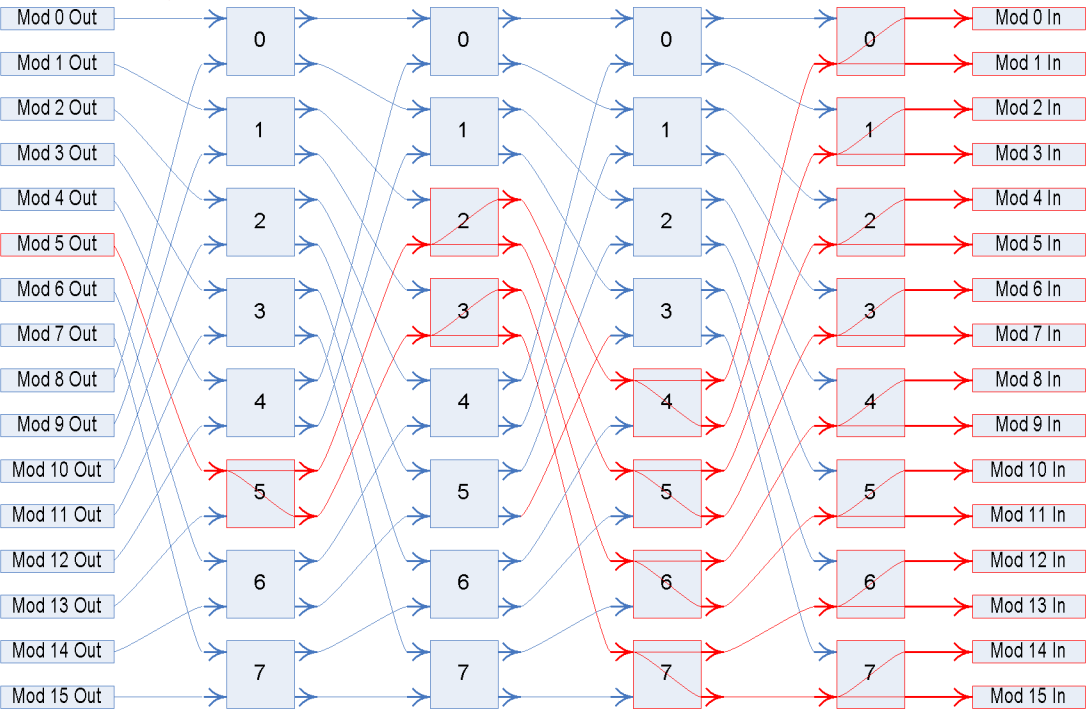
Triangle Grid	0xB_	0xC_	0xD_
0x_0	FORK, RF	ROT, RF	HOP, RF
0x_1	FORK, RR	ROT, RR	HOP, RR
0x_2	FORK, RL	ROT, RL	HOP, RL
0x_3	FORK, RB	ROT, RB	HOP, RB
0x_4	FORK, PF	ROT, PF	HOP, PF
0x_5	FORK, SR	ROT, SR	HOP, SR
0x_6	FORK, SL	ROT, SL	HOP, SL
0x_7	FORK, PB	ROT, PB	HOP, PB

Square Grid	0xB_	0xC_	0xD_
0x_0	FORK, RF	ROT, RF	HOP, RF
0x_1	FORK, RR	ROT, RR	HOP, RR
0x_2	FORK, RL	ROT, RL	HOP, RL
0x_3	FORK, RB	ROT, RB	HOP, RB
0x_4	FORK, AF	ROT, AF	HOP, AF
0x_5	FORK, AR	ROT, AR	HOP, AR
0x_6	FORK, AL	ROT, AL	HOP, AL
0x_7	FORK, AB	ROT, AB	HOP, AB

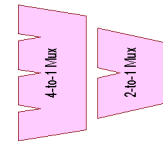
Example: Transfer initiated from module 2 to module 12:



Example: Broadcast connection initiated from module 5 (address byte sent with 0101 source address and broadcast bit set)



Module Addresses	Flag Update Byte Order	Universal Module Commands	Module-Specific Commands
0001 = Instruction Processor	For full flag update: order = FS3, FS2, FS1, FS0*	00000000 INIT initialization	10000*** COND,* conditional opcode (0-7) Conditional
0010 = Register Bank	For ¾ flag update: order = FS2, FS1, FS0*	00000001 EOI end of instruction	10000001 PCXY request PC XY Memory
0011 = Conditional & Flag Subsystem	For ½ update: order = FS1, FS0*	00000010 BTI broadcast data transfer to temp register.	10000010 PCD request PC direction fields Memory
0100 = ALU Subsystem	For ¼ update: order = FS0*	00000011 - unused	10000011 PCXYD request PC XY and direction fields Memory
0101 = Context Switcher	*FS0 packet will have EOT bit set.	00000100 HLT cpu fault with halt	
0110 = Program Counter Control & Memory		00000101 FLT cpu fault without halt	
0111 = Game Feature Subsystem		0001**** ASF,* auto-select flag (0-15)	
1000 =	FS3 = flags 12-15 X, R**, P**, T**	00100*** COND,* conditional opcode (0-7)	
1001 =	FS2 = flags 8-11 M, W, O, K	00101*** - unused (0-7)	
1010 =	FS1 = flags 4-7 V, C, A, B		
1011 =	FS0 = flags 0-3 D, F, Z, N		
1100 =	** Generated or static flags, update is ignored.		
1101 =	Byte format (bits 7-0): F3M,F3S,F2M,F2S,F1M,F1S,F0M,F0S		
1110 =	F*M are flag mask bits, F*S are flag state bits		
1111 =	A set flag mask bit indicates a flag should be updated with the state value.		



Network functionality:

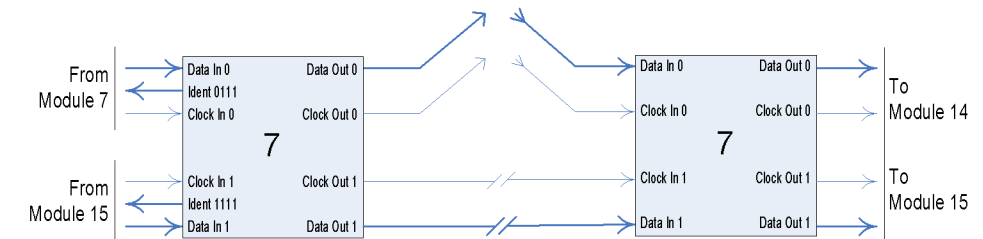
1. Transmission type "broadcast" plus connection type "stream":
 - 1a. Broadcast/stream packet type opens a persistent fanout connection on the network. The network links include bi-directional destination-to-source links that are logically OR'ed together and flow back to the broadcast source.
 - 1b. A source address can change the broadcast connection into a point-to-point by sending out a packet of connection type "collapse broadcast", with the destination or XOR tag in the address field. This effectively causes the nodes to recursively disconnect unnecessary links.
 - 1c. In stream modes, the "disconnect" connection type is used by the source to sever the stream link. The EOT flag is not an alternative to the "disconnect" connection functionality and is only for protocol-level functionality.

2. Transmission type "packet":

- 2a. Broadcasts and "single-destination" connection types are allowed, but "collapse broadcast" and "disconnect" are not used.
- 2b. An ident packet is a link packet type with an XOR-tag address.
- 2c. An EOT flag is primarily used to latch registers or as an application-level end of communication. It can be used with any of the packet types, although its functionality with the "link" packet type is ignored.

3. Address tag:

- 3a. Broadcast will always specify source address in address field.
- 3b. "Disconnect stream" packets do not use addressing.
- 3b. Everything else will always use XOR tag addressing.



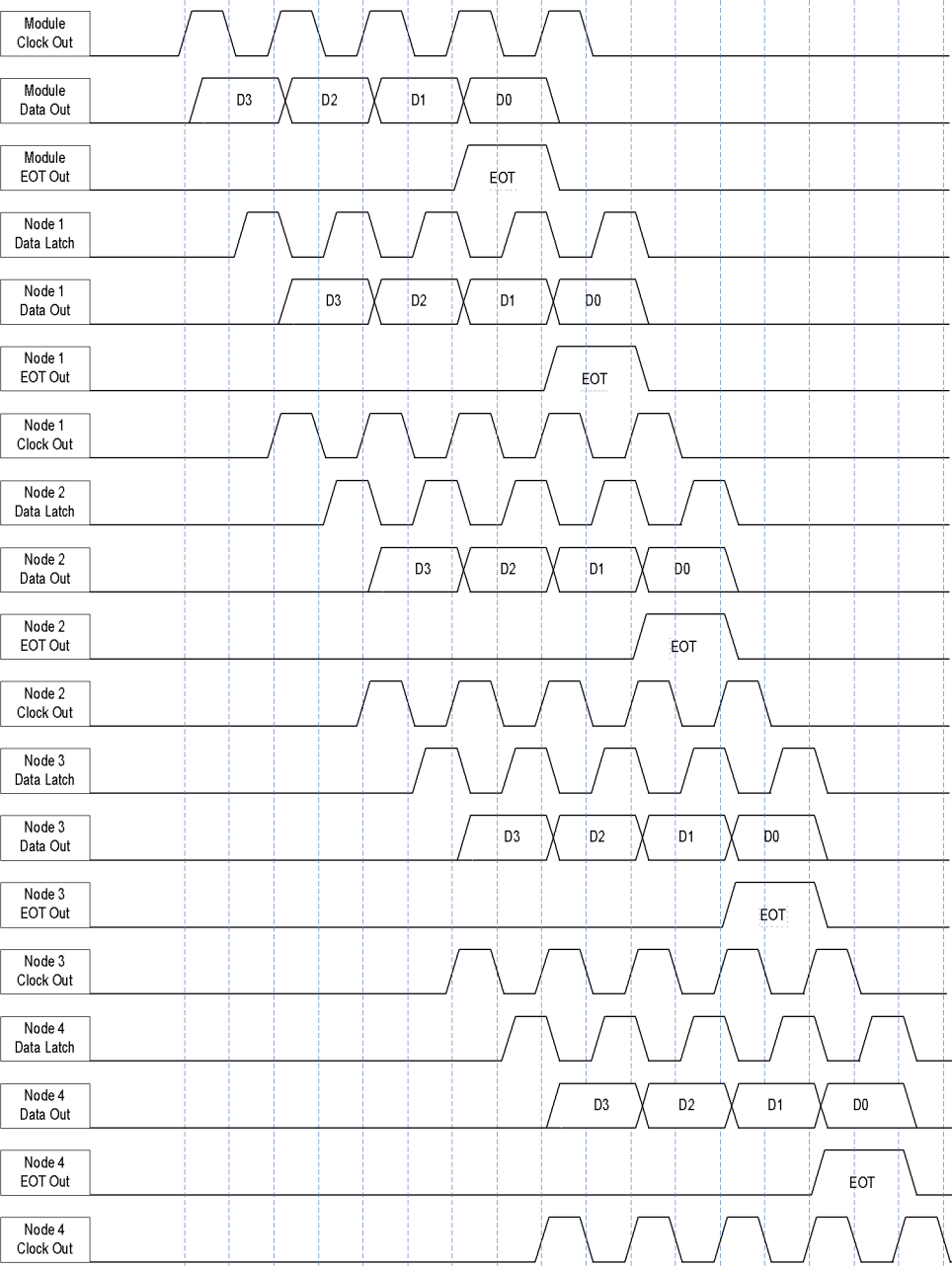
Packet transfers:

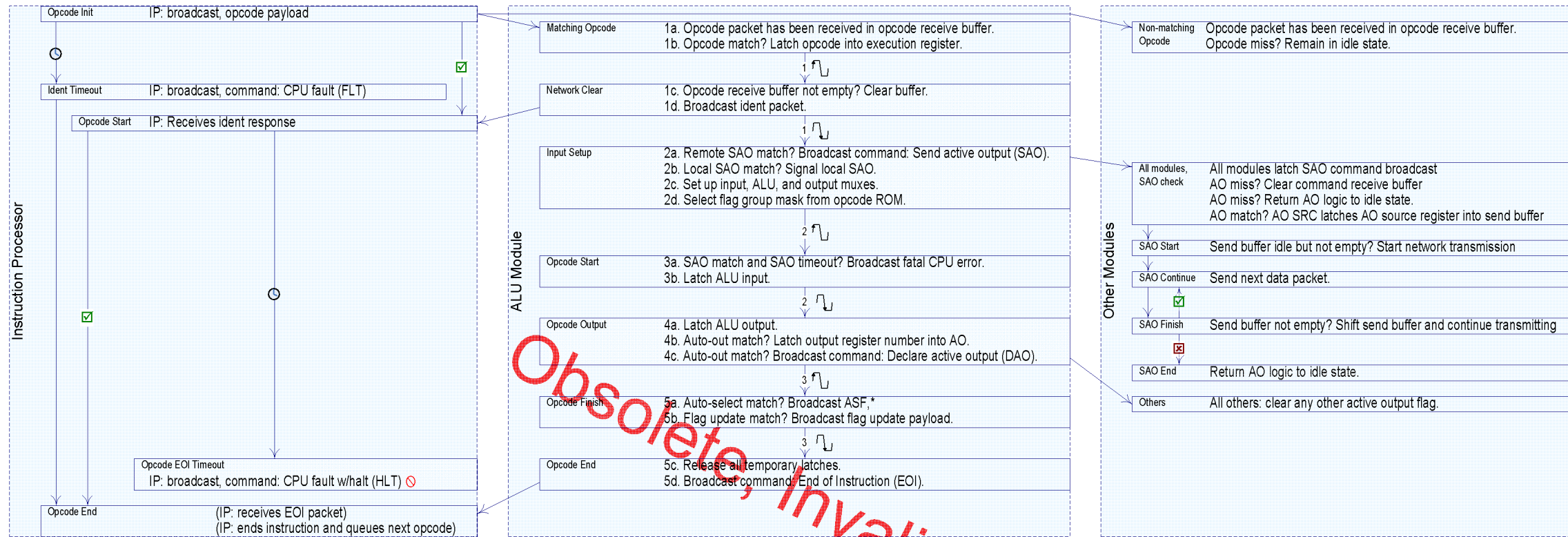
1. Data/address word is presented to the data bus of the first network node.
2. On the first rising clock edge, the data is latched on the first node.
3. Once the data is latched, the routing logic sets up the multiplexers for further distribution of the packet.
4. Once the multiplexers are established, the data/address word flows through the enabled multiplexer(s) and onto the second node(s).
5. On the first falling clock edge, the data is latched on the second node and the first node latch is released.
6. Once the data is latched, the routing logic sets up the multiplexers for further distribution of the packet.
7. Once the multiplexers are established, the data/address word flows through the enabled multiplexer(s) and onto the third node(s).
8. On the rising clock edge, the data is latched on the third node and the second node latch is released.
9. Steps 3-8 are repeated for the third and fourth nodes.
10. On the third rising clock edge, the data is latched or shifted onto the module register specified by data bits 6 and 7 and the EOT bit state is latched. The fourth node latch is released.
11. If more data needs to be transferred, the next trailing clock edge signals that the data should be presented to the network data bus and steps 1-10 are repeated. The final clock edge should have the EOT bit set.

Packet-based network: packet layout
b16: Transmission type: 0=single-destination, 1=broadcast
b12-b15: XOR Address (non-broadcast) or Source address (broadcast)
b9-b11: Packet type
000=Link packet
001=Data packet
010=Command packet
011=Flag update
100=Opcode packet
101=Program Counter packet
110=Extended Process Info. Packet
111= <i>unused</i>
b8: EOT flag
b0-b7: Single-byte payload

b0-b7: Link packet payload:
b7: initiate stream connection 1=yes, 0=no
b6: disconnect stream 1=yes 0=no
b5: collapse broadcast 1=yes 0=no
b4: stream uses receive clock 1=yes 0=no
b0-b3: Aux. address field.

Return stream path:
b8: EOT flag
b0-7: Single-byte payload





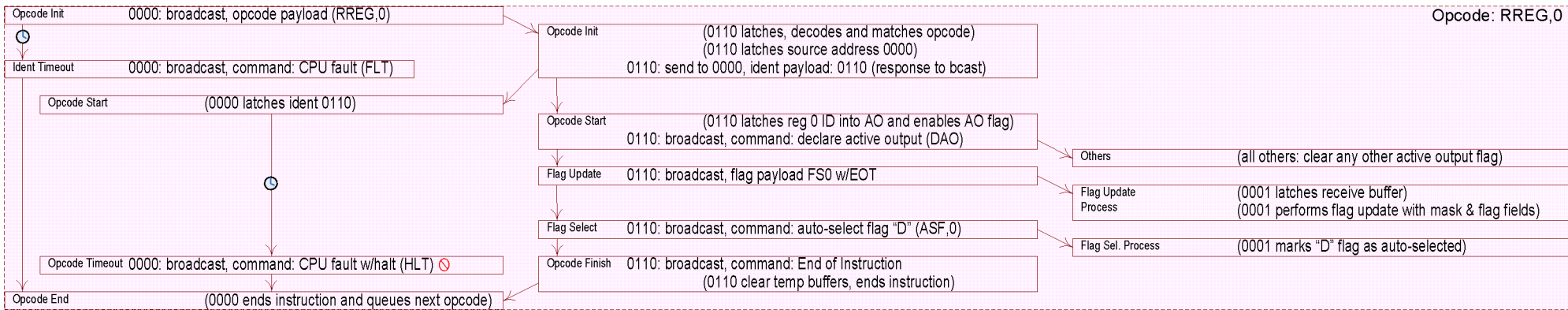
1. Opcode packet and source has been latched.
1a. Decode opcode and check for match.
1b. Upon match, transmit ident packet to source.

2. Next clock cycle:
2a. Latch opcode into execution register.
2b. Flag if SAO is required.
2ba: Broadcast command; Send active output (SAO).
2c. Set up input, ALU, and output muxes.
2d. Flag if auto-out is required.
2e. Select flag group mask from opcode rom.
2f. Flag if auto-select-flag is needed.

3. Next clock cycle:
3a. Broadcast fault if SAO timeout and halt.
3b. Latch ALU input.

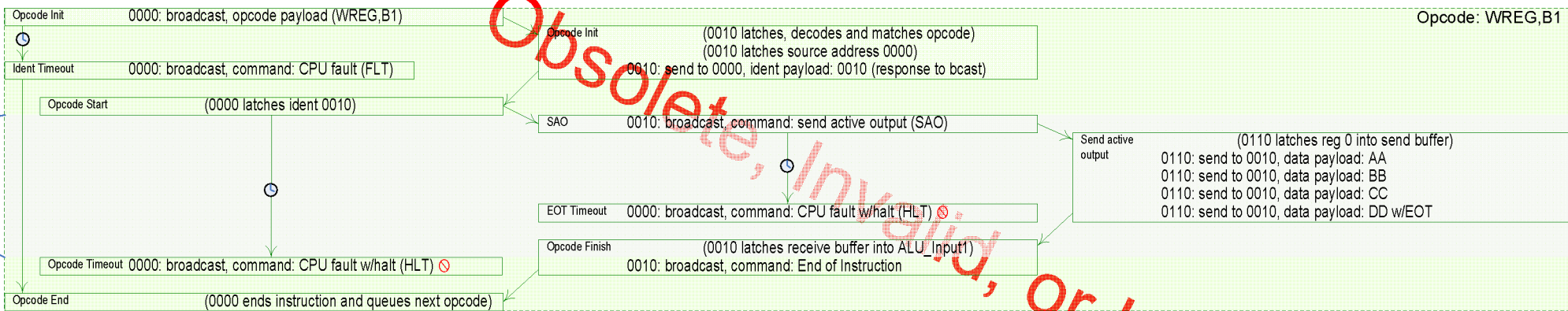
4. Next clock cycle:
4a. Latch ALU output
4b. If AO flag is enabled:
4ba: latch output register number into AO
4bb: Broadcast command: Declare active output (DAO).
5. If ASF is valid:
5a. Broadcast command: ASF,*
6. If flag update is required:
6a. Broadcast flag update byte 1.
6b. Broadcast flag update byte 2 w/ if EOI.
7. Broadcast command: End of instruction (EOI).

Example opcode microcode execution flow with direct register transfers. Opcodes: RREG,0; WREG,B1; INC
(Read register GP0, write to ALU Input B1, perform increment, select ALU output B0 for output)



1. Instruction Processor broadcasts opcode to modules.
- 1a. All modules decode opcode and check for matches.
- 1b. Upon match, register module records source address.
- 1c. Register module notifies broadcaster of opcode accept.

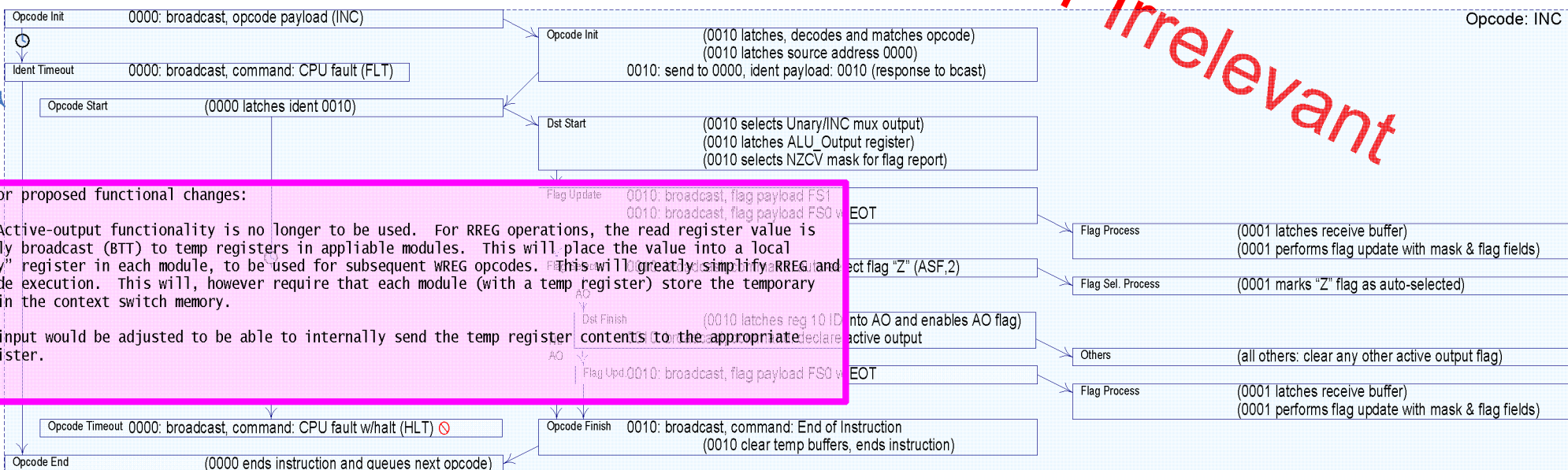
- 1d. Broadcaster records identity of register module.
2. Register module records register 0 as active output.
- 2a. Register module broadcasts active output declaration.
- 2b. All other modules clear any prior active output settings.
3. Register module broadcasts update of "D" flag status.
- 3a. Flag subsystem receives status.
- 3b. Flag subsystem updates "D" flag.
- 3c. Register module broadcasts "D" flag as auto-selected.
- 3d. Flag subsystem marks "D" flag as auto-selected.
4. Register module broadcasts end of instruction.
- 4a. Register module performs clean up.
- 4b. Instruction Processor queues up next opcode.



5. Instruction Processor broadcasts opcode to modules.
- 5a. All modules decode opcode and check for matches.
- 5b. Upon match, ALU module records source address.
- 5c. ALU module notifies broadcaster of opcode accept.

- 5d. Broadcaster records identity of ALU module.
6. ALU module broadcasts send active output request.
- 6a. Register module prepares register 0 data to be sent.
- 6b. Register module transmits uppermost byte.
- 6c. Register module transmits second upper byte.
- 6d. Register module transmits second lower byte.
- 6e. Register module transmits lowermost byte with EOT flag.

7. ALU module transfers received data into ALU Input1
- 7a. ALU module broadcasts end of instruction.
- 7b. Instruction Processor queues up next opcode.



8. Instruction Processor broadcasts opcode to modules.
- 8a. All modules decode opcode and check for matches.
- 8b. Upon match, ALU module records source address.
- 8c. Register module notifies broadcaster of opcode accept.

- 8d. Broadcaster records identity of ALU module.
9. ALU sets up multiplexers to perform INC math operation.
- 9a. ALU transfers INC math result into ALU output register.
- 9b. ALU selects mask of flags affected by math operation.

10. ALU broadcasts flag update of N and Z flags status.
- 10a. ALU broadcasts flag update of C and V flags status.
- 10b. Flag subsystem receives status.
- 10c. Flag subsystem updates requested flags.
- 10d. ALU module broadcasts "Z" flag as auto-selected.
- 10e. Flag subsystem marks "Z" flag as auto-selected.

11. ALU module records register 0 as active output.
- 11a. ALU module broadcasts active output declaration.
- 11b. All other modules clear any prior active output settings.
12. Register module broadcasts update of "D" flag status.
- 12a. Flag subsystem receives status.
- 12b. Flag subsystem updates "D" flag.
13. ALU module broadcasts end of instruction.
- 13a. ALU module performs cleanup.
- 13b. Instruction Processor queues up next opcode.

Note: Major proposed functional changes:

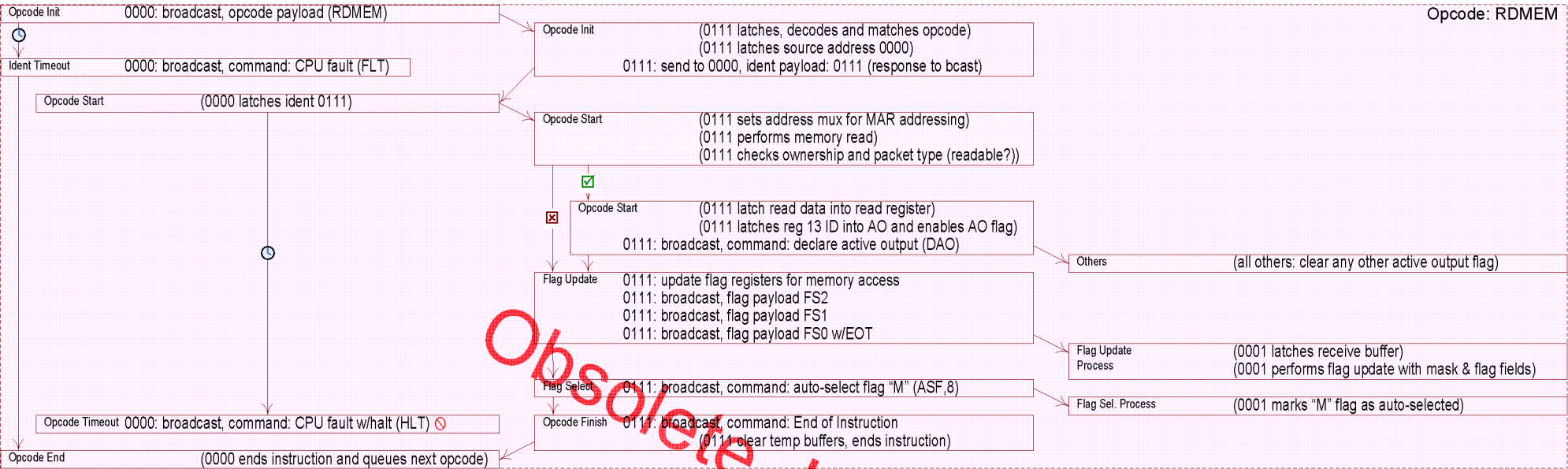
The 'AO' Active-output functionality is no longer to be used. For RREG operations, the read register value is immediately broadcast (BTT) to temp registers in applicable modules. This will place the value into a local "temporary" register in each module, to be used for subsequent WREG opcodes. This will greatly simplify RREG and WREG opcode execution. This will, however require that each module (with a temp register) store the temporary register in the context switch memory.

ALU auto-input would be adjusted to be able to internally send the temp register contents to the appropriate input register.

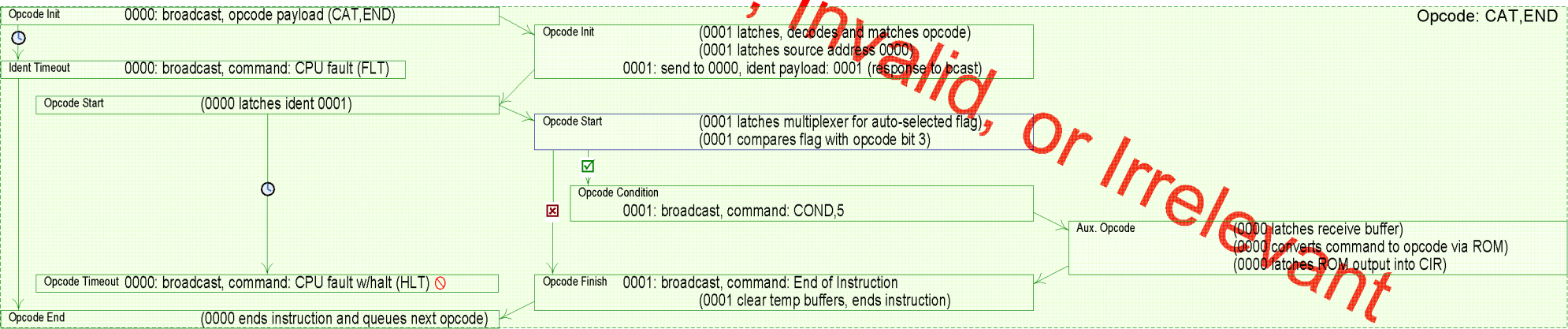
* If the appropriate "input auto-source" was enabled in the ALU configuration (b12-13=01), then these steps would run as part of the ALU INC opcode, and this WREG,B1 opcode would not be necessary.

** If "auto-output" was disabled in the ALU configuration, then an additional RREG,10 opcode would be needed to perform these steps.

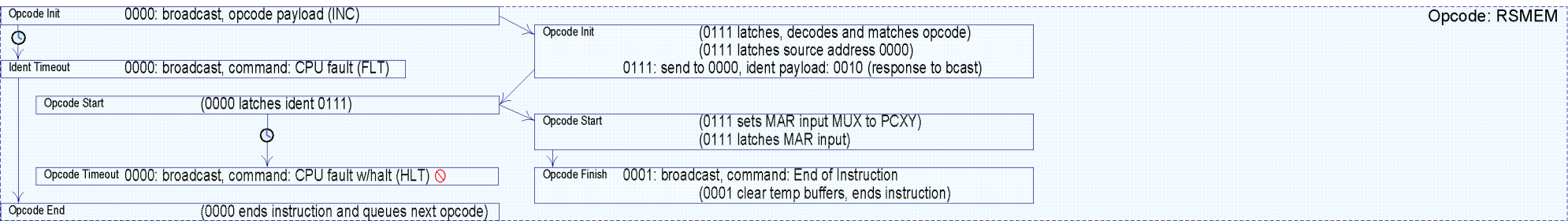
Example opcode microcode execution flow. Opcodes: RDMEM; CAT,END; RSMEM
(Read memory location; Run conditional END opcode if auto-selected flag is true; reset memory address register to PC location if not)



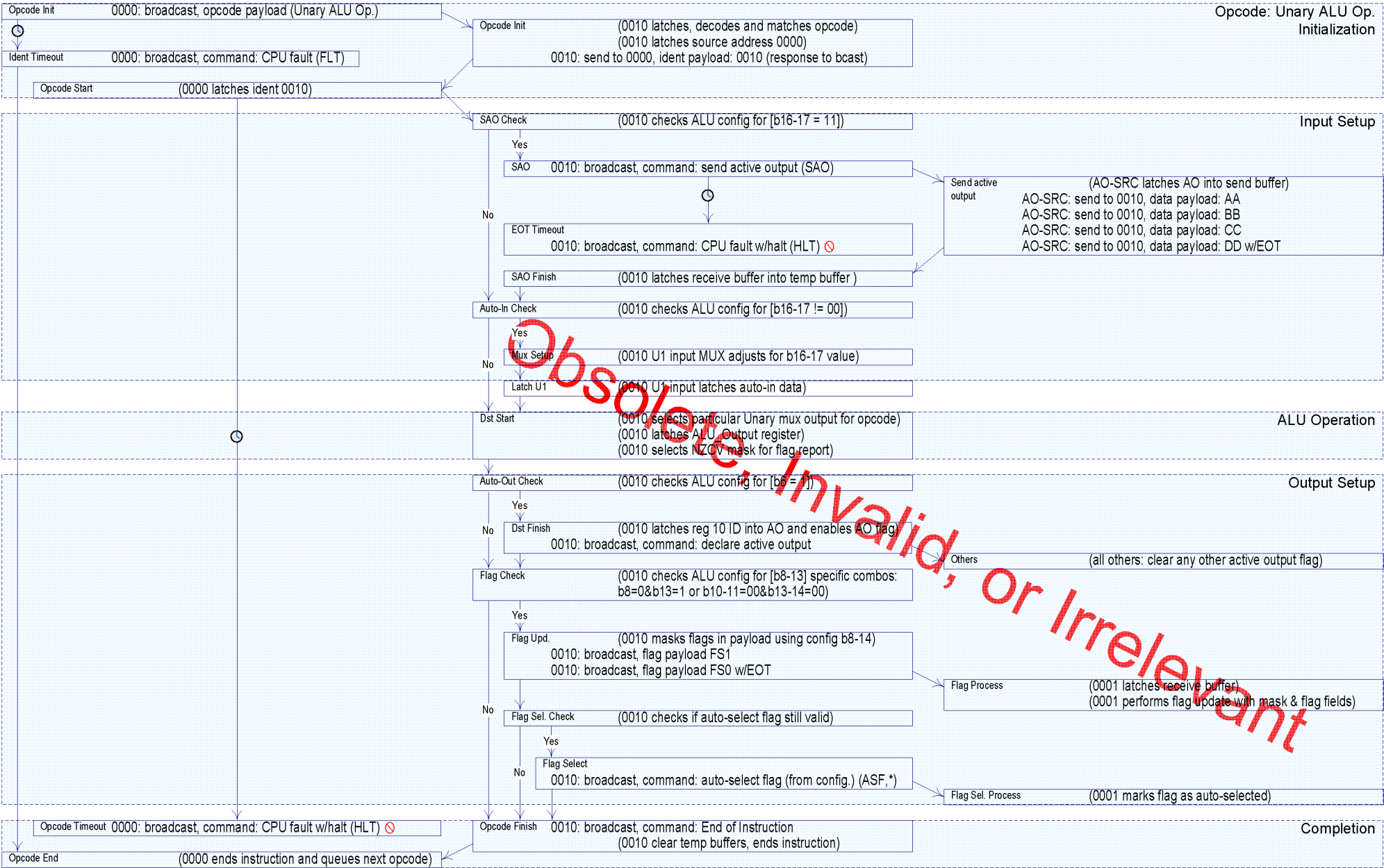
1. Instruction Processor broadcasts opcode to modules.
- 1a. All modules decode opcode and check for matches.
- 1b. Upon match, memory module records source address.
- 1c. Memory module notifies broadcaster of opcode accept.
- 1d. Broadcaster records identity of register module.
2. Memory module sets up multiplexers for MAR addressing
- 2a. Memory location is read.
- 2b. Extended data is checked for allowed access
- 2c. Data value is latched into read register.
3. Memory module records memory register as active output.
- 3a. Memory module broadcasts active output declaration.
- 3b. All other modules clear any prior active output settings.
4. Memory module updates flag registers for new access.
- 4a. Memory module broadcasts update of M/W/O flags status.
- 4b. Memory module broadcasts second flag byte.
- 4c. Memory module broadcasts first flag byte with EOT flag.
- 4d. Flag subsystem receives status.
- 4e. Flag subsystem updates flags M, W, and O.
- 4f. Memory module broadcasts "M" flag as auto-selected.
- 4g. Flag subsystem marks "M" flag as auto-selected.
5. Memory module broadcasts end of instruction.
- 5a. Memory module performs clean up.
- 6b. Instruction Processor queues up next opcode.



7. Instruction Processor broadcasts opcode to modules.
- 7a. All modules decode opcode and check for matches.
- 7b. Upon match, flag module records source address.
- 7c. Register module notifies broadcaster of opcode accept.
- 7d. Broadcaster records identity of flag module.
8. Flag module sets multiplexers for auto-selected flag.
- 8a. Flag module compares flag with T/F bit from opcode.
- 8b. If flag is true, flag module transmits END command.
9. Instruction processor receives command.
- 9a. I.P. converts conditional command to opcode value.
- 9b. I.P. transfers opcode into conditional instruction register.
10. I.P. module broadcasts end of instruction.
- 10a. I.P. module performs clean up.
- 10b. Instruction Processor queues up 'END' or next opcode.



11. Instruction Processor broadcasts opcode to modules.
- 11a. All modules decode opcode and check for matches.
- 11b. Upon match, ALU module records source address.
- 11c. Register module notifies broadcaster of opcode accept.
- 11d. Broadcaster records identity of ALU module.
12. Memory module selects PCXY reg for MAR input MUX
- 12a. Memory module transfers PCXY to MAR
13. ALU module broadcasts end of instruction.
- 13a. ALU module performs cleanup.
- 13b. Instruction Processor queues up next opcode.



1. Instruction Processor broadcasts opcode to modules.
1a. All modules decode opcode and check for matches.
1b. Upon match, ALU module records source address.
1c. Register module notifies broadcaster of opcode accept.

1d. Broadcaster records identity of ALU module.

2. ALU module checks ALU_U1 AI config for AO setting.

2a. ALU module broadcasts send active output request.
2b. Register module prepares register 0 data to be sent.
2c. Register module transmits uppermost byte.
2d. Register module transmits second upper byte.
2e. Register module transmits second lower byte.
2f. Register module transmits lowermost byte with EOT flag.

2g. ALU module transfers received data into temp buffer.

3. ALU module checks ALU_U1 AI config setting for nonzero.

3a. ALU module sets input MUX for auto-in source.

3b. ALU_U1 latches auto-in data.

4. ALU sets up multiplexers to perform math operation.
4a. ALU transfers math result into ALU output register.
4b. ALU selects mask of flags affected by math operation.

5. ALU module checks ALU_UO for auto-out enable setting.

5a. ALU module records register 0 as active output.
5b. ALU module broadcasts active output declaration.
5c. All other modules clear any prior active output settings.
6. ALU module checks ALU_U1 flag config for allowed flags.

6a. ALU module uses ALU_U1 flag config to mask flags.
6b. ALU module broadcasts flag update byte 1.
6c. ALU module broadcasts flag update byte 2 with EOT.
6d. Flag subsystem receives status.
6e. Flag subsystem updates specified flag.
6f. Check if ALU config still allows auto-select mask.

6g. ALU broadcasts flag as auto-selected based on config.
6h. Flag subsystem marks flag as auto-selected.

7. ALU module broadcasts end of instruction.
7a. ALU module performs cleanup.
7b. Instruction Processor queues up next opcode.

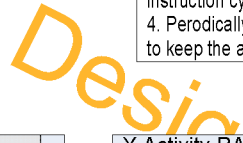
Error

[illegible]

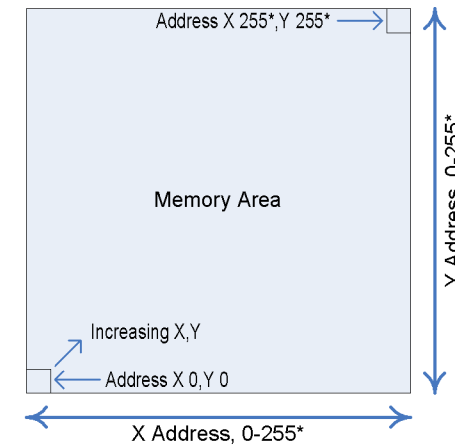
Pattern 11: 0000000 0000000 0000000 0011100 0111110 1111111 1111111 1111111 0010100 0

b8-11: Scan pattern

Game feature concept: Military Roles			
Process role:	Process role description:	CPU specialties:	Restrictions/limitations:
0000 Commander	Complex processing & more flexibility	Underlay/Overlay process DUPE process scheduling Ignores own user's write protection Underlay procs FORK overlay procs at MAR address	Banned: SCAN/ACT/MINE, WPM, WEM Overlay command procs cannot FORK command procs Difficulty in overwriting enemy cells
0001 Engineer	Defensive protections & obstacles	Create corrupt and planar cells Auto-write protection when writing Ignores own user's write protection	Banned: DUPE, SCAN/ACT/MINE; restricted FORK Cannot overwrite enemy cells
0010 Soldier	Offensive combat & map control	Fewer and weaker strong faults	Banned: DUPE, SCAN/ACT/MINE; restricted FORK
0011 Quartermaster	Resource gathering specialty	Easier glitch location, faster mining; MINE opcode	Banned: DUPE, WPM; restricted FORK
0100 Scout	Scan map for enemy assets & report	Quicker SCAN opcodes DUPE process scheduling	Banned: MINE, WPM, WEM, FORK
0101 Spy	Steal info from enemy processes	Can grab PC info from enemy process Less jamming risk Enemy process termination upon success	Banned: MINE, DUPE, WEM, FORK Locked movement & direction once special ability activated
0110 E-warfare	Prevent or corrupt enemy communication	Jam enemy processes ability to use group resources Corrupt group resources	Banned: MINE, DUPE, WPM, WEM, FORK Locked movement & direction once special ability activated
0111 Demolition	Break map blockages	Immunity from strong faults Ability to quickly remove corrupt/planar cells MINE opcode	Banned: DUPE, SCAN/ACT, WPM, FORK, JMPMA
1000 Defense	Protect interface	DUPE process scheduling, Fewer and weaker strong faults Easier enemy cell overwriting	Banned: MINE, FORK; Movement can only occur for first instruction cycle, then movement locked
1001 Tank	Punch through enemy cells	Fewer and weaker strong faults MINE opcode	Banned: DUPE, SCAN/ACT, WPM



- How the users are arranged on the actual physical memory map.
- How each user "sees" their own view of the memory map.
- * The X and Y sizes of the memory map are each adjustable to one of 16, 32, 64, 128, or 256 addresses.
- Conceptual Stage – Not Implemented



How each user “sees” their own view of the memory map.

How the users are arranged on the actual physical memory map.

How each user "sees" their own view of the memory map.

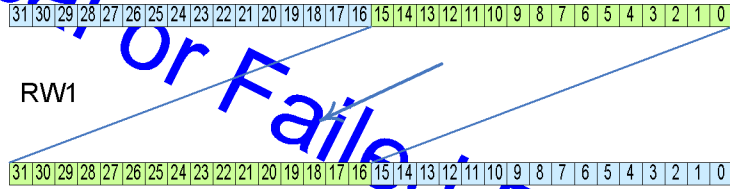
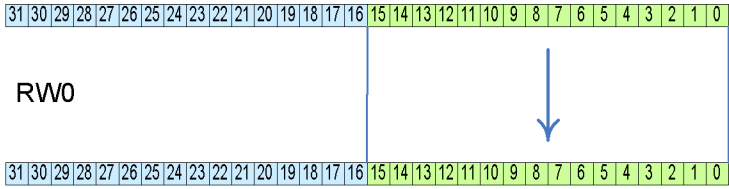
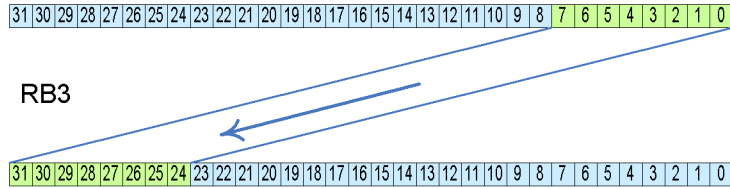
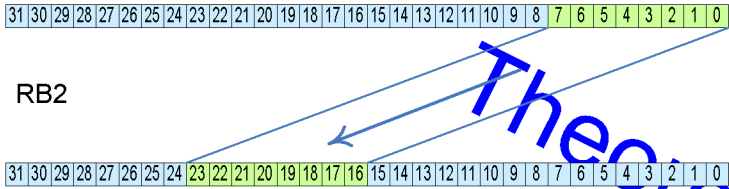
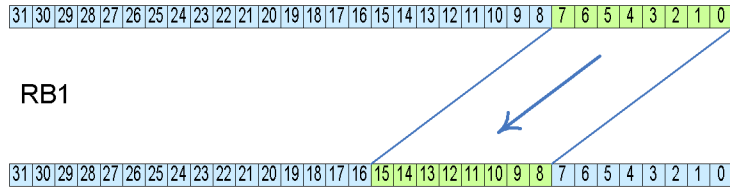
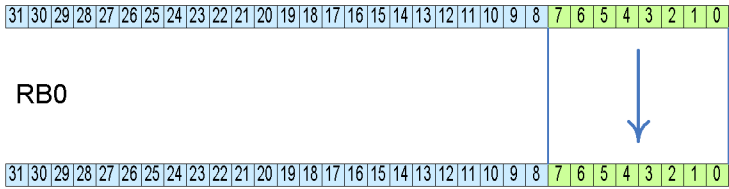
- * The X and Y sizes of the memory map are each adjustable to one of 16, 32, 64, 128, or 256 addresses.

Conceptual Stage – Not Implemented

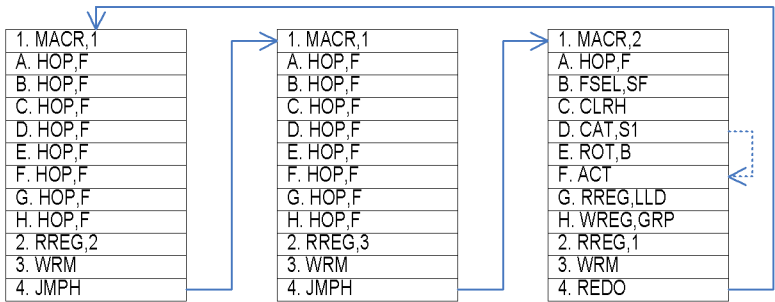
Y Activity RAM Location 15	Y:240-255
...	...
...	...
...	...
...	...
...	...
...	...
...	...
Main Map	...
...	...
...	...
...	...
Y Activity RAM Location 3	Y:48-63
Y Activity RAM Location 2	Y:32-47
Y Activity RAM Location 1	Y:16-31
Y Activity RAM Location 0	Y:0-15



Activity reported to
process at 188,170:
X activity: 11:10
Y activity: 10:7



Theoretical or Failed Designs – Not Planned



Reg 1:	Reg 2:	Reg 3:	Macro 1:	Macro 2:
1. MACR,1	1. MACR,1	1. MACR,2	A. HOP,F	A. HOP,F
2. RREG,2	2. RREG,3	2. RREG,1	B. HOP,F	B. FSEL,SF
3. WRM	3. WRM	3. WRM	C. HOP,F	C. CLRH
4. JMPH	4. JMPH	4. JMPH	D. HOP,F	D. CAT,S1
			E. HOP,F	E. ROT,B
			F. HOP,F	F. ACT
			G. HOP,F	G. RREG,LLD
			H. HOP,F	H. WREG,GRP

